Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М80-211БВ-25

Студент: Князев Александр Владимирович

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.10.26

Москва, 2026

Постановка задачи

Вариант 6

Разработать и отладить две программы на языке Си для Linux, реализующие взаимодействие родительского и дочернего процессов через именованную разделяемую память. За основу берётся лабораторная работа №1. Канал передачи результатов заменяется отображением shared memory в адресные пространства процессов, а чтение и запись согласуются именованными семафорами.

Первой строкой пользователь вводит имя входного файла. Родитель открывает его на чтение и перенаправляет стандартный ввод дочернего процесса на этот файл. В каждой строке записаны целые числа типа int; их количество произвольно. Ребёнок вычисляет отдельную сумму для каждой строки и передаёт её родителю. Родитель выводит полученные суммы в стандартный поток вывода.

В первой лабораторной работе 22 варианта. Для номера 6 в списке группы вариант определяется выражением ((6 - 1) % 22) + 1 = 6. В третьей лабораторной сохраняется тот же вариант.

Необходимо обрабатывать системные ошибки, освобождать созданные ресурсы и формировать отдельные имена общей памяти и семафоров для каждой пары процессов. Для обмена используется объект shm_open, а не обычный файл на диске.

Общий метод и алгоритм решения

Организация процессов и данных

Проект содержит parent.c, child.c, shared.h, CMakeLists.txt и входной файл numbers.txt. Родитель создаёт один дочерний процесс через fork и запускает отдельную программу child через execve. Схема движения данных: входной файл → child → разделяемая память → parent → терминал.

Общая структура shared_data_t определена в shared.h и содержит тип сообщения type и сумму sum типа long long. Входные числа проверяются на соответствие диапазону int. Более широкий тип суммы позволяет получить, например, 2147483648 при сложении 2147483647 и 1. В проверенной среде sizeof(shared_data_t) составляет 16 байт; программа использует sizeof, а не фиксированное число.

Создание общей памяти

Родитель формирует имена на основе своего PID и времени запуска, создаёт объект через shm_open с флагами O_CREAT | O_EXCL | O_RDWR и правами 0600, задаёт размер через ftruncate и выполняет mmap с MAP_SHARED. Ребёнок после execve самостоятельно открывает тот же объект по имени, проверяет его размер и создаёт своё отображение.

Разделяемая память обозначает общий объект данных, а memory mapping - способ подключить этот объект к адресному пространству процесса. Указатели parent и child могут различаться, но записи относятся к одним и тем же данным. Представление объекта в /dev/shm является штатной реализацией POSIX shared memory в Linux на tmpfs [1].

Синхронизация чтения и записи

Используются два именованных семафора: empty с начальным значением 1 и ready с начальным значением 0. Ребёнок вызывает sem_wait(empty), записывает сообщение и выполняет sem_post(ready). Родитель ожидает ready, копирует оба поля структуры в локальные переменные и только после этого выполняет sem_post(empty). Так ячейка не перезаписывается до копирования результата.

Имена shared memory, empty и ready передаются ребёнку через аргументы execve. Состояние семафоров определяет, когда можно обращаться к ячейке; отдельный volatile или цикл постоянного опроса флага не используется. При нулевом значении sem_wait блокирует вызывающий процесс [2].

Ввод и завершение обмена

Ребёнок читает входной файл через read блоками по 4096 байт. Состояние разбора числа сохраняется между чтениями, поэтому строка и число могут пересекать границу буфера. Обрабатываются знаки, пробелы, табуляция и окончания строк CRLF. Пустые строки пропускаются; последняя непустая строка вычисляется и без завершающего перевода строки. Сложность разбора линейна по объёму входа, рабочий буфер имеет постоянный размер.

Для суммы отправляется MESSAGE_RESULT, после успешного конца файла - MESSAGE_DONE. При ошибке входных данных ребёнок пишет причину в stderr и передаёт MESSAGE_ERROR. Родитель ожидает ready через sem_timedwait с интервалом проверки в одну секунду и при тайм-ауте проверяет ребёнка через waitpid с WNOHANG. Если ребёнок завершился, дополнительно проверяется последнее ожидающее сообщение через sem_trywait.

При обработанной ошибке родитель останавливает ещё работающего ребёнка, затем получает его статус. Ребёнок закрывает собственные семафоры и отображение; удаление имён через sem_unlink и shm_unlink выполняет родитель. Гарантия очистки относится к штатному завершению и обработанным ошибкам; внешнее принудительное уничтожение родителя этим протоколом не обрабатывается.

Код программы

Ниже приведены полные актуальные исходники, проверенные при подготовке отчёта. В обеих программах используется _POSIX_C_SOURCE 200809L для объявления нужных интерфейсов POSIX.

shared.h

```
#ifndef OS_LAB3_SHARED_H
#define OS_LAB3_SHARED_H

typedef enum {
    MESSAGE_RESULT = 1,
    MESSAGE_DONE,
    MESSAGE_ERROR
} message_type_t;

typedef struct {
    message_type_t type;
    long long sum;
} shared_data_t;

#endif
```

parent.c

```
#define _POSIX_C_SOURCE 200809L

#include "shared.h"

#include <errno.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <stdio.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

extern char **environ;

typedef struct {
    int fd;
    shared_data_t *data;
    sem_t *empty;
    sem_t *ready;
    char shm_name[128];
    char empty_name[128];
    char ready_name[128];
} ipc_t;

static int write_all(int fd, const char *buffer, size_t length){
    size_t sent = 0;
    while (sent < length) {
        ssize_t n = write(fd, buffer + sent, length - sent);
        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n <= 0) {
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}
```

```
static void report_error(const char *message){
    size_t length = 0;
    while (message[length] != '\0') {
        length++;
    }
    (void)write_all(STDERR_FILENO, message, length);
}

static int print_number(long long value){
    char reversed[3 * sizeof(long long) + 3];
    size_t length = 0;
    long long current = value;
    do {
        int digit = (int)(current % 10);
        if (digit < 0) {
            digit = -digit;
        }
        reversed[length++] = (char)('0' + digit);
        current /= 10;
    } while (current != 0);
    if (value < 0) {
        reversed[length++] = '-';
    }

    char output[sizeof(reversed)];
    for (size_t i = 0; i < length; i++) {
        output[i] = reversed[length - 1 - i];
    }
    output[length] = '\n';
    return write_all(STDOUT_FILENO, output, length + 1);
}

static int create_ipc(ipc_t *ipc){
    struct timespec now;
    if (clock_gettime(CLOCK_REALTIME, &now) == -1) {
        perror("Cannot get time for IPC names");
        return -1;
    }

    char prefix[96];
    int length = snprintf(prefix, sizeof(prefix), "/os_lab3_%ld_%lld_%ld",
                          (long)getpid(), (long long)now.tv_sec, now.tv_nsec);
    if (length < 0 || (size_t)length >= sizeof(prefix)) {
        report_error("Cannot generate IPC names\n");
        return -1;
    }
    int shm_length = snprintf(ipc->shm_name, sizeof(ipc->shm_name),
                              "%s_shm", prefix);
    int empty_length = snprintf(ipc->empty_name, sizeof(ipc->empty_name),
                                "%s_empty", prefix);
    int ready_length = snprintf(ipc->ready_name, sizeof(ipc->ready_name),
                                "%s_ready", prefix);
    if (shm_length < 0 || (size_t)shm_length >= sizeof(ipc->shm_name) ||
        empty_length < 0 || (size_t)empty_length >= sizeof(ipc->empty_name) ||
        ready_length < 0 || (size_t)ready_length >= sizeof(ipc->ready_name)) {
        report_error("Cannot generate IPC names\n");
        return -1;
    }

    ipc->fd = shm_open(ipc->shm_name, O_CREAT | O_EXCL | O_RDWR, 0600);
    if (ipc->fd == -1) {
        perror("Cannot create shared memory");
        return -1;
    }
    if (ftruncate(ipc->fd, (off_t)sizeof(shared_data_t)) == -1) {
        perror("Cannot set shared memory size");
```

```
        return -1;
    }
    ipc->data = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE,
                     MAP_SHARED, ipc->fd, 0);
    if (ipc->data == MAP_FAILED) {
        perror("Cannot map shared memory");
        return -1;
    }

    ipc->empty = sem_open(ipc->empty_name, O_CREAT | O_EXCL, 0600, 1);
    if (ipc->empty == SEM_FAILED) {
        perror("Cannot create empty semaphore");
        return -1;
    }
    ipc->ready = sem_open(ipc->ready_name, O_CREAT | O_EXCL, 0600, 0);
    if (ipc->ready == SEM_FAILED) {
        perror("Cannot create ready semaphore");
        return -1;
    }
    return 0;
}

static int destroy_ipc(ipc_t *ipc){
    int failed = 0;
    if (ipc->ready != SEM_FAILED) {
        if (sem_unlink(ipc->ready_name) == -1) {
            perror("Cannot unlink ready semaphore");
            failed = 1;
        }
        if (sem_close(ipc->ready) == -1) {
            perror("Cannot close ready semaphore");
            failed = 1;
        }
    }
    if (ipc->empty != SEM_FAILED) {
        if (sem_unlink(ipc->empty_name) == -1) {
            perror("Cannot unlink empty semaphore");
            failed = 1;
        }
        if (sem_close(ipc->empty) == -1) {
            perror("Cannot close empty semaphore");
            failed = 1;
        }
    }
    if (ipc->data != MAP_FAILED &&
        munmap(ipc->data, sizeof(shared_data_t)) == -1) {
        perror("Cannot unmap shared memory");
        failed = 1;
    }
    if (ipc->fd != -1) {
        if (shm_unlink(ipc->shm_name) == -1) {
            perror("Cannot unlink shared memory");
            failed = 1;
        }
        if (close(ipc->fd) == -1) {
            perror("Cannot close shared memory descriptor");
            failed = 1;
        }
    }
    return failed ? -1 : 0;
}

static int wait_for_message(sem_t *ready, pid_t pid, int *status, int *reaped){
    while (1) {
        struct timespec deadline;
        if (clock_gettime(CLOCK_REALTIME, &deadline) == -1) {
            return -1;
```

```
        }
        deadline.tv_sec++;

        int result;
        do {
            result = sem_timedwait(ready, &deadline);
        } while (result == -1 && errno == EINTR);
        if (result == 0) {
            return 0;
        }
        if (errno != ETIMEDOUT) {
            return -1;
        }

        if (!*reaped) {
            pid_t waited;
            do {
                waited = waitpid(pid, status, WNOHANG);
            } while (waited == -1 && errno == EINTR);
            if (waited == -1) {
                return -1;
            }
            *reaped = waited == pid;
        }
        if (*reaped) {
            do {
                result = sem_trywait(ready);
            } while (result == -1 && errno == EINTR);
            if (result == -1 && errno == EAGAIN) {
                errno = EPIPE;
            }
            return result;
        }
    }
}

int main(void){
    struct sigaction action = {0};
    action.sa_handler = SIG_IGN;
    if (sigemptyset(&action.sa_mask) == -1 ||
        sigaction(SIGPIPE, &action, NULL) == -1) {
        perror("Cannot configure SIGPIPE handling");
        return 1;
    }

    const char prompt[] = "Enter filename: ";
    if (write_all(STDERR_FILENO, prompt, sizeof(prompt) - 1) == -1) {
        return 1;
    }
    char filename[256];
    if (scanf("%255s", filename) != 1) {
        report_error("Cannot read filename\n");
        return 1;
    }
    int filefd = open(filename, O_RDONLY);
    if (filefd == -1) {
        perror("Cannot open input file");
        return 1;
    }

    ipc_t ipc = {
        .fd = -1,
        .data = MAP_FAILED,
        .empty = SEM_FAILED,
        .ready = SEM_FAILED
    };
    if (create_ipc(&ipc) == -1) {
```

```
    (void)destroy_ipc(&ipc);
    if (close(filefd) == -1) {
        perror("Cannot close input file");
    }
    return 1;
}

pid_t pid = fork();
if (pid == -1) {
    perror("Cannot fork");
    (void)destroy_ipc(&ipc);
    if (close(filefd) == -1) {
        perror("Cannot close input file");
    }
    return 1;
}
if (pid == 0) {
    if (dup2(filefd, STDIN_FILENO) == -1) {
        perror("Cannot redirect child stdin");
        _exit(1);
    }
    if (filefd != STDIN_FILENO && close(filefd) == -1) {
        perror("Cannot close child input file");
        _exit(1);
    }
    char *const args[] = {
        "./child", ipc.shm_name, ipc.empty_name, ipc.ready_name, NULL
    };
    execve("./child", args, environ);
    perror("Cannot execute child");
    _exit(127);
}

int failed = 0;
int reaped = 0;
int status = 0;
if (close(filefd) == -1) {
    perror("Cannot close input file");
    failed = 1;
}

while (!failed) {
    if (wait_for_message(ipc.ready, pid, &status, &reaped) == -1) {
        perror("Cannot receive child message");
        failed = 1;
        break;
    }

    message_type_t type = ipc.data->type;
    long long sum = ipc.data->sum;
    if (sem_post(ipc.empty) == -1) {
        perror("Cannot release message slot");
        failed = 1;
        break;
    }
    if (type == MESSAGE_DONE) {
        break;
    }
    if (type == MESSAGE_ERROR) {
        report_error("Child reported an error\n");
        failed = 1;
        break;
    }
    if (type != MESSAGE_RESULT) {
        report_error("Unknown child message type\n");
        failed = 1;
        break;
```

```
        }
        if (print_number(sum) == -1) {
            perror("Cannot write result");
            failed = 1;
        }
    }

    if (failed && !reaped && kill(pid, SIGTERM) == -1 && errno != ESRCH) {
        perror("Cannot stop child");
    }
    if (!reaped) {
        pid_t waited;
        do {
            waited = waitpid(pid, &status, 0);
        } while (waited == -1 && errno == EINTR);
        if (waited == -1) {
            perror("Cannot wait for child");
            failed = 1;
        } else {
            reaped = 1;
        }
    }
    if (reaped && (!WIFEXITED(status) || WEXITSTATUS(status) != 0)) {
        report_error("Child failed\n");
        failed = 1;
    }
    if (destroy_ipc(&ipc) == -1) {
        failed = 1;
    }
    return failed ? 1 : 0;
}
```

child.c

```
#define _POSIX_C_SOURCE 200809L

#include "shared.h"
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <semaphore.h>
#include <stdio.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

typedef struct {
    int fd;
    shared_data_t *data;
    sem_t *empty;
    sem_t *ready;
} ipc_t;

static int write_all(int fd, const char *buffer, size_t length){
    size_t sent = 0;
    while (sent < length) {
        ssize_t n = write(fd, buffer + sent, length - sent);
        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n <= 0) {
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}
```

```
static void report_error(const char *message){
    size_t length = 0;
    while (message[length] != '\0') {
        length++;
    }
    (void)write_all(STDERR_FILENO, message, length);
}

static int open_ipc(ipc_t *ipc, const char *shm_name, const char *empty_name,
const char *ready_name){
    ipc->fd = shm_open(shm_name, O_RDWR, 0);
    if (ipc->fd == -1) {
        perror("Cannot open shared memory");
        return -1;
    }

    struct stat info;
    if (fstat(ipc->fd, &info) == -1) {
        perror("Cannot get shared memory size");
        return -1;
    }
    if (info.st_size != (off_t)sizeof(shared_data_t)) {
        report_error("Unexpected shared memory size\n");
        return -1;
    }

    ipc->data = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE,
                     MAP_SHARED, ipc->fd, 0);

    if (ipc->data == MAP_FAILED) {
        perror("Cannot map shared memory");
        return -1;
    }
    ipc->empty = sem_open(empty_name, 0);
    if (ipc->empty == SEM_FAILED) {
        perror("Cannot open empty semaphore");
        return -1;
    }
    ipc->ready = sem_open(ready_name, 0);
    if (ipc->ready == SEM_FAILED) {
        perror("Cannot open ready semaphore");
        return -1;
    }
    return 0;
}

static int close_ipc(ipc_t *ipc) {
    int failed = 0;
    if (ipc->ready != SEM_FAILED && sem_close(ipc->ready) == -1) {
        perror("Cannot close ready semaphore");
        failed = 1;
    }
    if (ipc->empty != SEM_FAILED && sem_close(ipc->empty) == -1) {
        perror("Cannot close empty semaphore");
        failed = 1;
    }
    if (ipc->data != MAP_FAILED &&
        munmap(ipc->data, sizeof(shared_data_t)) == -1) {
        perror("Cannot unmap shared memory");
        failed = 1;
    }
    if (ipc->fd != -1 && close(ipc->fd) == -1) {
        perror("Cannot close shared memory descriptor");
        failed = 1;
    }
    return failed ? -1 : 0;
}
```

```
static int send_message(ipc_t *ipc, message_type_t type, long long sum) {
    int result;
    do {
        result = sem_wait(ipc->empty);
    } while (result == -1 && errno == EINTR);
    if (result == -1) {
        perror("Cannot wait for message slot");
        return -1;
    }
    ipc->data->type = type;
    ipc->data->sum = sum;
    if (sem_post(ipc->ready) == -1) {
        perror("Cannot notify parent");
        return -1;
    }
    return 0;
}

static int input_error(ipc_t *ipc, const char *message) {
    report_error(message);
    (void)send_message(ipc, MESSAGE_ERROR, 0);
    return -1;
}

static int process_input(ipc_t *ipc) {
    char buffer[4096];
    long long sum = 0;
    long long number = 0;
    int negative = 0;
    int in_number = 0;
    int has_digit = 0;
    int has_values = 0;

    while (1) {
        ssize_t n = read(STDIN_FILENO, buffer, sizeof(buffer));
        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n == -1) {
            perror("Cannot read input file");
            (void)send_message(ipc, MESSAGE_ERROR, 0);
            return -1;
        }

        int end_of_file = n == 0;
        ssize_t count = end_of_file ? 1 : n;

        for (ssize_t i = 0; i < count; i++) {
            char c = end_of_file ? '\n' : buffer[i];

            if (c >= '0' && c <= '9') {
                if (!in_number) {
                    in_number = 1;
                    number = 0;
                    negative = 0;
                    has_digit = 0;
                }

                int digit = c - '0';
                long long limit = negative ? -(long long)INT_MIN : INT_MAX;
                if (number > (limit - digit) / 10) {
                    return input_error(ipc, "Input number is outside int
range\n");
                }
                number = number * 10 + digit;
                has_digit = 1;
```

```
            } else if (c == '-' || c == '+') {
                if (in_number) {
                    return input_error(ipc, "Invalid sign in input number\n");
                }
                in_number = 1;
                number = 0;
                has_digit = 0;
                negative = c == '-';
            } else if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
                if (in_number) {
                    if (!has_digit) {
                        return input_error(ipc, "Input sign has no digits\n");
                    }
                    long long value = negative ? -number : number;
                    if ((value > 0 && sum > LLONG_MAX - value) ||
                        (value < 0 && sum < LLONG_MIN - value)) {
                        return input_error(ipc, "Sum is outside long long
range\n");
                    }
                    sum += value;
                    has_values = 1;
                    in_number = 0;
                    has_digit = 0;
                }
                if (c == '\n' && has_values) {
                    if (send_message(ipc, MESSAGE_RESULT, sum) == -1) {
                        return -1;
                    }
                    sum = 0;
                    has_values = 0;
                }
            } else {
                return input_error(ipc, "Invalid character in input file\n");
            }
        }
        if (end_of_file) {
            return 0;
        }
    }
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        report_error("Usage: ./child <shared_memory> <empty_semaphore>
<ready_semaphore>\n");
        return 1;
    }

    ipc_t ipc = {
        .fd = -1,
        .data = MAP_FAILED,
        .empty = SEM_FAILED,
        .ready = SEM_FAILED
    };
    if (open_ipc(&ipc, argv[1], argv[2], argv[3]) == -1) {
        (void)close_ipc(&ipc);
        return 1;
    }
    int failed = process_input(&ipc) == -1;
    if (!failed && send_message(&ipc, MESSAGE_DONE, 0) == -1) {
        failed = 1;
    }
    if (close_ipc(&ipc) == -1) {
        failed = 1;
    }
    return failed ? 1 : 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(OC_Lab3 C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

set(THREADS_PREFER_PTHREAD_FLAG ON)
find_package(Threads REQUIRED)

add_executable(parent parent.c shared.h)
add_executable(child child.c shared.h)

target_link_libraries(parent PRIVATE Threads::Threads rt)
target_link_libraries(child PRIVATE Threads::Threads rt)

add_dependencies(parent child)

set_target_properties(parent child PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}"
)
```

Протокол работы программы

Среда и сборка

Проверки выполнены 30.10.2026 на Ubuntu 24.04 LTS x86_64. Компилятор: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0. Использовался cmake version 3.28.3. Для диагностики сборка выполнена с -Wall -Wextra -Wpedantic -Werror; статический анализ - с -fanalyzer. Четыре граничных сценария дополнительно проверены с UndefinedBehaviorSanitizer.

При ручной сборке из папки проекта используются команды:

```
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    parent.c -o parent -pthread -lrt
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    child.c -o child -pthread -lrt
```

CMake собирает те же две программы и при запросе цели parent предварительно собирает child. RUNTIME_OUTPUT_DIRECTORY указывает на папку исходников:

```
cmake -S . -B cmake-build-debug
cmake --build cmake-build-debug --target parent
```

Для запуска parent, child и numbers.txt находятся в одной папке. Имя файла вводится без пробелов, не более 255 символов. Родитель запускает ребёнка по относительному пути ./child.

Основной пример

Содержимое numbers.txt:

```
10 20 30
5 15 25
100 200 300
-10 5 3
2147483647 1
```

Запуск и ввод имени файла:

```
./parent
Enter filename: numbers.txt
```

Фактический стандартный вывод:

```
60
45
600
-2
2147483648
```

Подсказка Enter filename выводится в stderr, а суммы - в stdout. Последняя сумма превышает INT_MAX, но корректно помещается в long long.

Трассировка взаимодействия процессов

Трассировка получена при фактическом запуске с numbers.txt с помощью strace 6.8. Ключ -f включает дочерний процесс; фильтр оставляет создание процессов, операции с памятью и файлами, синхронизацию и завершение. Для воспроизведения из папки проекта:

```
events=clone,clone3,fork,vfork,execve,openat,close,dup2
events=$events,ftruncate,mmap,munmap,link,unlink
events=$events,futex,wait4,exit_group,read,write
strace -f -s 180 -e trace="$events" -o trace.txt ./parent
```

После запуска вводится numbers.txt. Переменная events задаёт один список системных вызовов для фильтра strace. Полная фактическая трассировка находится в materials/strace-ipc.txt. Ниже показаны выбранные строки; длинные имена и отдельные аргументы сокращены многоточием. Родитель имеет PID 37424, ребёнок - 37425.

Создание памяти и дочернего процесса

```
37424 openat(AT_FDCWD, "/dev/shm/os_lab3_37424_..._shm",
             O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC, 0600) = 4
37424 ftruncate(4, 16) = 0
37424 mmap(NULL, 16, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0)
      = 0x7935bdd3e000
37424 clone(child_stack=NULL,
            flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
            child_tidptr=...) = 37425
37425 dup2(3, 0) = 0
37425 execve("./child", ["./child", "/os_lab3_37424_..._shm",
             "/os_lab3_37424_..._empty", "/os_lab3_37424_..._ready"],
             ...) = 0
37425 openat(AT_FDCWD, "/dev/shm/os_lab3_37424_..._shm",
             O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 3
37425 mmap(NULL, 16, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)
      = 0x7c4ca4c18000
```

В этой среде fork реализован библиотекой через системный вызов clone. В показанных флагах отсутствуют CLONE_THREAD и CLONE_VM: создаётся отдельный процесс. dup2 подключает открытый входной файл к дескриптору 0, а execve загружает программу child. Оба процесса отображают один именованный объект длиной 16 байт; виртуальные адреса 0x7935bdd3e000 и 0x7c4ca4c18000 различаются.

Семафоры и ожидание готовности

```
37424 link("/dev/shm/sem.nZsBVJ",
           "/dev/shm/sem.os_lab3_37424_..._empty") = 0
37424 link("/dev/shm/sem.8CG7fs",
           "/dev/shm/sem.os_lab3_37424_..._ready") = 0
37424 futex(0x7935bdd3c000,
            FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0,
            {tv_sec=1793363977, tv_nsec=301427035},
            FUTEX_BITSET_MATCH_ANY <unfinished ...>
37425 futex(0x7c4ca4c16000, FUTEX_WAKE, 1 <unfinished ...>
37424 <... futex resumed>) = 0
37425 <... futex resumed>) = 1
37424 write(1, "60\n", 3 <unfinished ...>
```

sem_open и sem_post являются библиотечными функциями, поэтому их имена не обязаны появляться в strace. В этой трассировке создание именованных семафоров видно через файлы sem.* и link, а блокирующее ожидание и пробуждение - через futex. sem_timedwait родителя ожидает ready, а sem_post ребёнка пробуждает ожидающий процесс. Это наблюдаемая реализация в данной среде, а не отдельный вызов futex из исходного кода [2, 3].

Завершение и удаление объектов

```
37425 munmap(0x7c4ca4c18000, 16) = 0
37425 exit_group(0) = ?
37425 +++ exited with 0 +++
37424 <... wait4 resumed>
      [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 37425
37424 unlink("/dev/shm/sem.os_lab3_37424_..._ready") = 0
37424 unlink("/dev/shm/sem.os_lab3_37424_..._empty") = 0
37424 munmap(0x7935bdd3e000, 16) = 0
37424 unlink("/dev/shm/os_lab3_37424_..._shm") = 0
37424 +++ exited with 0 +++
```

Ребёнок снимает своё отображение и завершается с кодом 0. Родитель получает статус ребёнка через waitpid, который в трассе представлен wait4, затем удаляет имена семафоров и общей памяти. Снятие отображения munmap и удаление имени shm_unlink выполняют разные задачи; в трассе удаление имени отображается как unlink [1].

Вывод

Переработана программа первой лабораторной работы для варианта 6. Дочерний процесс читает целые числа из файла и вычисляет сумму каждой строки, а родитель получает результаты через именованную разделяемую память и выводит их в терминал. Для согласования доступа используется пара именованных семафоров, а для каждого запуска формируются отдельные имена объектов.